

cpp/ Competitive Programming Notebook

Contents

1	Competitive Programming Template	1
2	Customizable Lazy Segment Tree	2
3	Li Chao Tree / Convex Hull Trick	3
4	Grid Transformations & Utilities	4
5	Text Manipulation & Regex	6

1 Competitive Programming Template

```
91     return 0;
92 }
```

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  #ifdef LOCAL
5  #   define de(...)
6  std::println(std::cout,
7              "[{} = ({})]",
8  #   __VA_ARGS__,
9              std::make_tuple(__VA_ARGS__))
10
11 #   define deb(x) cout << #x << '=' << x << '\n';
12 #else
13 #   define de(...) ((void)0)
14 #endif
15
16 // --- Type Aliases ---
17 using ll = long long;
18 using ull = unsigned long long;
19 using ld = long double;
20 using pii = pair<int, int>;
21 using pll = pair<ll, ll>;
22 using vi = vector<int>;
23 using vl = vector<ll>;
24 using vpi = vector<pii>;
25 using vpl = vector<pll>;
26 using vvi = vector<vi>;
27 using vvl = vector<vl>;
28
29 template <class T>
30 using vec = vector<T>;
31
32 template <class T>
33 using minpq = priority_queue<T, vector<T>, greater<T>>;
34
35 // --- Constants ---
36 constexpr int INF = 1e9 + 7;
37 constexpr ll LINF = 4e18;
38 constexpr int MOD = 1e9 + 7;
39 constexpr int MOD9 = 998244353;
40
41 vvi d4 = {{-1, 0}, // U
42          {0, 1}, // R
43          {1, 0}, // D
44          {0, -1}}; // L
45 vvi d8 = {{-1, 0}, // U
46          {-1, 1}, // UR
47          {0, 1}, // R
48          {1, 1}, // DR
49          {1, 0}, // D
50          {1, -1}, // DL
51          {0, -1}, // L
52          {-1, -1}}; // UL
53
54 // --- Macros & Shortcuts ---
55 #define all(x) begin(x), end(x)
56 #define rall(x) rbegin(x), rend(x)
57 #define sz(x) (int)(x).size()
58 #define pb push_back
59 #define fo(i, n) for (int i = 0; i < (int)(n); ++i)
60 #define foi(i, a, b) for (int i = (int)(a); i < (int)(b)
61     ⇨ ; ++i)
62 #define fod(i, a, b) for (int i = (int)(a); i > (int)(b)
63     ⇨ ; --i)
64
65 // --- Utility Functions ---
66 template <class T, class U>
67 inline bool chmin(T &a, const U &b) {
68     return b < a ? a = b, 1 : 0;
69 }
70 template <class T, class U>
71 inline bool chmax(T &a, const U &b) {
72     return a < b ? a = b, 1 : 0;
73 }
74
75 template <class T>
76 istream &operator>>(istream &is, vector<T> &v) {
77     for (auto &x : v)
78         is >> x;
79     return is;
80 }
81
82 void solve() {
83
84 int main() {
85     cin.tie(nullptr)->sync_with_stdio(false);
86
87     int t = 1;
88     // cin >> t;
89     while (t--) {
90         solve();
91     }
92 }
```

2 Customizable Lazy Segment Tree

Customization Recipes:

Query	IDENTITY	combine(a,b)	apply_lazy (Set)
Sum (RSQ)	0	a + b	st[p] = val * len
Min (RMQ)	2e9 / 4e18	min(a,b)	st[p] = val
Max (RMaxQ)	-2e9 / -4e18	max(a,b)	st[p] = val
GCD	0	std::gcd(a,b)	st[p] = val

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // =====
5 // Generic Lazy Segment Tree (0-indexed)
6 //
7 // HOW TO CUSTOMIZE FOR DIFFERENT QUERIES:
8 // -----
9 // 1. Range Sum (RSQ):
10 // IDENTITY = 0;
11 // combine(a, b) = a + b;
12 // apply(p, L, R, val):
13 // st[p] = val * (R - L + 1); (set)
14 // st[p] += val * (R - L + 1); (add)
15 //
16 // 2. Range Min (RMQ):
17 // IDENTITY = INF; (2e9 or 4e18)
18 // combine(a, b) = min(a, b);
19 // apply(p, L, R, val):
20 // st[p] = val; (set)
21 // st[p] += val; (add)
22 //
23 // 3. Range Max (RMaxQ):
24 // IDENTITY = -INF; (-2e9 or -4e18)
25 // combine(a, b) = max(a, b);
26 // apply(p, L, R, val):
27 // st[p] = val; (set)
28 // st[p] += val; (add)
29 //
30 // 4. Special Query (e.g. {min, count}):
31 // Use a struct with custom combine.
32 // =====
33
34 template <class T = long long>
35 class SegmentTree {
36 private:
37     int n;
38     vector<T> A, st, lazy;
39     vector<bool> has_lazy;
40
41     // ===== CUSTOMIZE HERE =====
42     static constexpr T IDENTITY = 0; // RSQ: 0 | RMQ: 2e9
43     // | RMaxQ: -2e9
44
45     T combine(T a, T b) const {
46         // RSQ: a + b | RMQ: min | RMaxQ: max
47         return a + b;
48     }
49
50     void apply_lazy(int p, int L, int R, T val) {
51         // Range Assignment:
52         st[p] = val * (R - L + 1);
53         lazy[p] = val;
54         has_lazy[p] = true;
55
56         // (Alternative: Range Addition):
57         // st[p] += val * (R - L + 1);
58         // lazy[p] += val;
59         // has_lazy[p] = true;
60     }
61
62     int l(int p) { return p << 1; }
63     int r(int p) { return (p << 1) + 1; }
64
65     void push(int p, int L, int R) {
66         if (!has_lazy[p] || L == R) return;
67         int m = (L + R) / 2;
68         apply_lazy(l(p), L, m, lazy[p]);
69         apply_lazy(r(p), m + 1, R, lazy[p]);
70         has_lazy[p] = false;
71         lazy[p] = 0;
72     }
73
74     void build(int p, int L, int R) {
75         if (L == R) {
76             st[p] = A[L];
77             return;
78         }
79         int m = (L + R) / 2;
80         build(l(p), L, m);
81         build(r(p), m + 1, R);
82         st[p] = combine(st[l(p)], st[r(p)]);
83     }
84
85     void update(int p, int L, int R, int i, int j, T val)
86         <= {
87         if (i > j || L > j || R < i) return;

```

```

87         if (L >= i && R <= j) {
88             apply_lazy(p, L, R, val);
89             return;
90         }
91         push(p, L, R);
92         int m = (L + R) / 2;
93         update(l(p), L, m, i, j, val);
94         update(r(p), m + 1, R, i, j, val);
95         st[p] = combine(st[l(p)], st[r(p)]);
96     }
97
98     T query(int p, int L, int R, int i, int j) {
99         if (i > j || L > j || R < i) return IDENTITY;
100         if (L >= i && R <= j) return st[p];
101         push(p, L, R);
102         int m = (L + R) / 2;
103         return combine(query(l(p), L, m, i, j),
104             query(r(p), m + 1, R, i, j));
105     }
106
107 public:
108     SegmentTree(int sz)
109         : n(sz), A(n, 0), st(4 * n, IDENTITY),
110           lazy(4 * n, 0), has_lazy(4 * n, false) {}
111
112     SegmentTree(const vector<T> &initialA)
113         : n((int)initialA.size()), A(initialA),
114           st(4 * n, IDENTITY), lazy(4 * n, 0),
115           has_lazy(4 * n, false) {
116         build(1, 0, n - 1);
117     }
118
119     void update(int i, int j, T val) { update(1, 0, n - 1,
120         <= i, j, val); }
121
122     void update(int idx, T val) { update(1, 0, n - 1,
123         <= idx, idx, val); }
124
125     T query(int i, int j) { return query(1, 0, n - 1, i, j
126         <= ); }
127
128     T RSQ(int i, int j) { return query(i, j); }
129     T RMQ(int i, int j) { return query(i, j); }
130
131 int main() {
132     vector<long long> A = {18, 17, 13, 19, 15, 11, 20,
133         <= 99}; // n = 8
134     SegmentTree<long long> st(A);
135
136     printf("      idx      0, 1, 2, 3, 4, 5,
137         <= 6, 7\n");
138     printf("      A is {18, 17, 13, 19, 15, 11,
139         <= 20, 99}\n");
140     printf("RSQ(1, 3) = %lld\n", st.query(1, 3)); //
141         <= 17+13+19 = 49
142     printf("RSQ(4, 7) = %lld\n", st.query(4, 7)); //
143         <= 15+11+20+99 = 145
144     printf("RSQ(3, 4) = %lld\n", st.query(3, 4)); // 19+15
145         <= 34
146
147     st.update(5, 5, 77); // update A[5] to 77
148     printf("\nNow, modify A into {18, 17, 13, 19, 15, 77,
149         <= 20, 99}\n");
150     printf("RSQ(1, 3) = %lld\n", st.query(1, 3)); // 49 (
151         <= unchanged)
152     printf("RSQ(4, 7) = %lld\n", st.query(4, 7)); //
153         <= 15+77+20+99 = 211
154     printf("RSQ(3, 4) = %lld\n", st.query(3, 4)); // 34 (
155         <= unchanged)
156
157     st.update(0, 3, 30); // update A[0..3] to 30
158     printf("\nNow, modify A into {30, 30, 30, 30, 15, 77,
159         <= 20, 99}\n");
160     printf("RSQ(1, 3) = %lld\n", st.query(1, 3)); //
161         <= 30+30+30 = 90
162     printf("RSQ(4, 7) = %lld\n", st.query(4, 7)); // 211 (
163         <= unchanged)
164     printf("RSQ(3, 4) = %lld\n", st.query(3, 4)); // 30+15
165         <= 45
166
167     st.update(3, 3, 7); // update A[3] to 7
168     printf("\nNow, modify A into {30, 30, 30, 7, 15, 77,
169         <= 20, 99}\n");
170     printf("RSQ(1, 3) = %lld\n", st.query(1, 3)); //
171         <= 30+30+7 = 67
172     printf("RSQ(4, 7) = %lld\n", st.query(4, 7)); // 211 (
173         <= unchanged)
174     printf("RSQ(3, 4) = %lld\n", st.query(3, 4)); // 7+15
175         <= 22
176
177     return 0;
178 }

```

3 Li Chao Tree / Convex Hull Trick

Convex Hull Trick (CHT) Optimization:

Maintains lower envelope of lines $y = mx + c$ in $O(\log X)$. Optimizes:

$$dp[i] = \min_{j < i} (m_j \cdot x_i + c_j) + \text{extra}(i)$$

Quadratic Cost (e.g. Frog 3): $dp[i] = \min_j \{dp[j] + (h_i - h_j)^2 + C\}$

$$\implies m = -2h_j, \quad c = dp[j] + h_j^2, \quad x = h_i, \quad \text{extra} = h_i^2 + C$$

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // =====
5 // Li Chao Segment Tree / Convex Hull Trick
6 //
7 // WHAT IT IS USED FOR:
8 // -----
9 // Maintains a dynamic envelope of linear
10 // functions (lines  $y = m * x + c$ ) and supports:
11 // 1. Inserting lines in ANY order
12 // (no monotonic slope requirement!)
13 // 2. Querying min (or max) at any  $x$ 
14 // in  $O(\log(\text{MAX}_X - \text{MIN}_X))$  time.
15 //
16 // DP OPTIMIZATION PATTERN:
17 // -----
18 // Solves transitions of the form:
19 //  $dp[i] = \min_{j < i} (m_j * x_i + c_j) + \text{extra}(i)$ 
20 // Reduces  $O(N^2)$  brute force to  $O(N \log(\text{range}))$ .
21 //
22 // EXAMPLE (Squared Distance / Frog 3):
23 //  $dp[i] = \min_{j < i} [dp[j] + (h[i] - h[j])^2 + C$ 
24 //  $\hookrightarrow j]$ 
25 //  $dp[i] = (h[i]^2 + C) + \min_{j < i} [(-2 * h[j]) * h[i]$ 
26 //  $\hookrightarrow j] + (dp[j] + h[j]^2)]$ 
27 //  $\rightarrow$  Slope:  $m = -2 * h[j]$ 
28 //  $\rightarrow$  Intercept:  $c = dp[j] + h[j]^2$ 
29 //  $\rightarrow$  Query:  $x = h[i]$ 
30 //  $\rightarrow$  Extra:  $\text{extra} = h[i]^2 + C$ 
31 // =====
32
33 using ll = long long;
34 constexpr ll LINF = 4e18; // Use -LINF for max queries
35
36 struct Line {
37     ll m, c;
38     ll eval(ll x) const { return m * x + c; }
39 };
40
41 // Li Chao Tree for MINIMUM queries
42 // (Flip '<' to '>' and min() to max() for MAXIMUM
43 // queries)
44
45 class LiChaoTree {
46 private:
47     struct Node {
48         Line line;
49         int left = -1, right = -1;
50         Node(Line l) : line(l) {}
51     };
52
53     ll min_x, max_x;
54     vector<Node> tree;
55
56     static constexpr Line DEFAULT_LINE = {0, LINF};
57
58     int get_left(int u) {
59         if (tree[u].left == -1) {
60             tree[u].left = (int)tree.size();
61             tree.emplace_back(DEFAULT_LINE);
62         }
63         return tree[u].left;
64     }
65
66     int get_right(int u) {
67         if (tree[u].right == -1) {
68             tree[u].right = (int)tree.size();
69             tree.emplace_back(DEFAULT_LINE);
70         }
71         return tree[u].right;
72     }
73
74     void add_line(int u, ll l, ll r, Line new_line) {
75         ll mid = l + (r - l) / 2;
76         bool mid_better = new_line.eval(mid) < tree[u].line.
77              $\hookrightarrow$  eval(mid); // '<' for min, '>' for max
78
79         if (mid_better) {
80             swap(tree[u].line, new_line);
81         }
82
83         if (l == r) return;
84     }

```

```

85     bool left_better = new_line.eval(l) < tree[u].line.
86          $\hookrightarrow$  eval(l); // '<' for min, '>' for max
87
88     if (left_better) {
89         int lc = get_left(u);
90         add_line(lc, l, mid, new_line);
91     } else {
92         int rc = get_right(u);
93         add_line(rc, mid + 1, r, new_line);
94     }
95 }
96
97 ll query(int u, ll l, ll r, ll x) {
98     if (u == -1) return LINF; // LINF for min, -LINF for
99      $\hookrightarrow$  max
100
101     ll res = tree[u].line.eval(x);
102     if (l == r) return res;
103
104     ll mid = l + (r - l) / 2;
105     if (x <= mid && tree[u].left != -1) {
106         res = min(res, query(tree[u].left, l, mid, x)); //
107          $\hookrightarrow$  min() for min, max() for max
108     } else if (x > mid && tree[u].right != -1) {
109         res = min(res, query(tree[u].right, mid + 1, r, x)
110              $\hookrightarrow$  );
111     }
112     return res;
113 }
114
115 public:
116     LiChaoTree(ll min_x = 0, ll max_x = 1'000'000'000)
117         : min_x(min_x), max_x(max_x) {
118         tree.emplace_back(DEFAULT_LINE);
119     }
120
121     // Insert a new line:  $y = m * x + c$ 
122     void add_line(ll m, ll c) {
123         add_line(0, min_x, max_x, {m, c});
124     }
125
126     // Query the minimum y value among all lines at x
127     ll query(ll x) {
128         return query(0, min_x, max_x, x);
129     }
130
131 int main() {
132     // --- Example 1: Basic Line Envelope ---
133     // Lines: L1:  $y=2x+10$ , L2:  $y=-x+25$ , L3:  $y=18$ 
134     LiChaoTree lct(0, 100);
135     lct.add_line(2, 10);
136     lct.add_line(-1, 25);
137     lct.add_line(0, 18);
138
139     printf("=== Example 1: Direct Envelope Queries ===\n")
140          $\hookrightarrow$  ;
141     for (ll x : {0, 5, 8, 10, 15}) {
142         printf("x = %2lld  $\rightarrow$  min y = %2lld\n", x, lct.query(
143              $\hookrightarrow$  x));
144     }
145
146     // --- Example 2: DP Optimization (Frog 3) ---
147     //  $dp[i] = \min_{j < i} (dp[j] + (h[i]-h[j])^2 + C)$ 
148     printf("\n=== Example 2: DP Optimization (Frog 3) ===\n")
149          $\hookrightarrow$  n");
150     int n = 5;
151     ll C = 6;
152     vector<ll> h = {1, 2, 4, 7, 9};
153     vector<ll> dp(n, 0);
154
155     // x-coordinate domain is [0, max(h)]
156     LiChaoTree dp_tree(0, 100);
157
158     // Insert base case:  $j = 0$ 
159     // Line:  $m = -2 * h[0]$ ,  $c = dp[0] + h[0]^2$ 
160     dp_tree.add_line(-2 * h[0], dp[0] + h[0] * h[0]);
161
162     for (int i = 1; i < n; ++i) {
163         // Query best line at  $x = h[i]$ 
164         ll best = dp_tree.query(h[i]);
165         dp[i] = best + h[i] * h[i] + C;
166
167         // Insert new line for future transitions from i
168         dp_tree.add_line(-2 * h[i], dp[i] + h[i] * h[i]);
169     }
170
171     for (int i = 0; i < n; ++i) {
172         printf("Stone %d (h=%2lld): dp = %lld\n", i, h[i],
173              $\hookrightarrow$  dp[i]);
174     }
175
176     return 0;
177 }

```

4 Grid Transformations & Utilities

Coordinate Transformations ($R \times C$):

Operation	Mapping $(r, c) \rightarrow$	Dimensions
Transpose	(c, r)	$C \times R$
Rotate 90° CW	$(c, R - 1 - r)$	$C \times R$
Rotate 90° CCW	$(C - 1 - c, r)$	$C \times R$
Rotate 180°	$(R - 1 - r, C - 1 - c)$	$R \times C$
Flip Horiz	$(r, C - 1 - c)$	$R \times C$
Flip Vert	$(R - 1 - r, c)$	$R \times C$
Anti-Diagonal	$r + c = k$	$0 \leq k < R + C - 1$
Main Diagonal	$r - c = k$	$-(C - 1) \leq k < R$

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // =====
5 // Grid & Matrix Transformation Cheat Sheet
6 //
7 // COORDINATE MAPPING SUMMARY (R x C):
8 // -----
9 // Original:      (r, c)
10 // Transpose:     (c, r)                [C x R]
11 // Rotate 90 CW:  (c, R - 1 - r)        [C x R]
12 // Rotate 90 CCW: (C - 1 - c, r)        [C x R]
13 // Rotate 180:    (R - 1 - r, C - 1 - c) [R x C]
14 // Flip Horiz:    (r, C - 1 - c)        [R x C]
15 // Flip Vert:     (R - 1 - r, c)        [R x C]
16 //
17 // DIAGONALS:
18 // -----
19 // Anti-diagonal (top-right to bot-left): r + c = const
20 // Main diagonal (top-left to bot-right):  r - c =
21 //                                     ↪ const
22 // =====
23 // --- 1. Transpose (R x C -> C x R) ---
24 template <class T>
25 vector<vector<T>> transpose(const vector<vector<T>> &g)
26     ↪ {
27     if (g.empty()) return {};
28     int R = g.size(), C = g[0].size();
29     vector<vector<T>> res(C, vector<T>(R));
30     for (int r = 0; r < R; ++r)
31         for (int c = 0; c < C; ++c)
32             res[c][r] = g[r][c];
33     return res;
34 }
35
36 vector<string> transpose(const vector<string> &g) {
37     if (g.empty()) return {};
38     int R = g.size(), C = g[0].size();
39     vector<string> res(C, string(R, ' '));
40     for (int r = 0; r < R; ++r)
41         for (int c = 0; c < C; ++c)
42             res[c][r] = g[r][c];
43     return res;
44 }
45
46 // --- 2. Rotate 90 Degrees Clockwise ---
47 template <class T>
48 vector<vector<T>> rotate90_cw(const vector<vector<T>> &g
49     ↪ ) {
50     if (g.empty()) return {};
51     int R = g.size(), C = g[0].size();
52     vector<vector<T>> res(C, vector<T>(R));
53     for (int r = 0; r < R; ++r)
54         for (int c = 0; c < C; ++c)
55             res[c][R - 1 - r] = g[r][c];
56     return res;
57 }
58
59 vector<string> rotate90_cw(const vector<string> &g) {
60     if (g.empty()) return {};
61     int R = g.size(), C = g[0].size();
62     vector<string> res(C, string(R, ' '));
63     for (int r = 0; r < R; ++r)
64         for (int c = 0; c < C; ++c)
65             res[c][R - 1 - r] = g[r][c];
66     return res;
67 }
68
69 // --- 3. Rotate 90 Degrees Counter-Clockwise ---
70 template <class T>
71 vector<vector<T>> rotate90_ccw(const vector<vector<T>> &
72     ↪ g) {
73     if (g.empty()) return {};
74     int R = g.size(), C = g[0].size();
75     vector<vector<T>> res(C, vector<T>(R));
76     for (int r = 0; r < R; ++r)
77         for (int c = 0; c < C; ++c)
78             res[C - 1 - c][r] = g[r][c];
79     return res;
80 }
81
82 // --- 4. Rotate 180 Degrees ---
83 template <class T>

```

```

81 vector<vector<T>> rotate180(const vector<vector<T>> &g)
82     ↪ {
83     if (g.empty()) return {};
84     int R = g.size(), C = g[0].size();
85     vector<vector<T>> res(R, vector<T>(C));
86     for (int r = 0; r < R; ++r)
87         for (int c = 0; c < C; ++c)
88             res[R - 1 - r][C - 1 - c] = g[r][c];
89     return res;
90 }
91
92 // --- 5. In-place Square Matrix Rotate 90 CW (O(1)
93     ↪ extra space) ---
94 template <class T>
95 void rotate_square_inplace(vector<vector<T>> &mat) {
96     int N = mat.size();
97     for (int i = 0; i < N; ++i)
98         for (int j = i + 1; j < N; ++j)
99             swap(mat[i][j], mat[j][i]); // Transpose
100     for (int i = 0; i < N; ++i)
101         reverse(mat[i].begin(), mat[i].end()); // Reverse
102     ↪ each row
103 }
104
105 // --- 6. Flips / Reflections ---
106 template <class T>
107 vector<vector<T>> flip_h(vector<vector<T>> g) { //
108     ↪ Horizontal flip (left <-> right)
109     for (auto &row : g) reverse(row.begin(), row.end());
110     return g;
111 }
112
113 template <class T>
114 vector<vector<T>> flip_v(vector<vector<T>> g) { //
115     ↪ Vertical flip (top <-> bottom)
116     reverse(g.begin(), g.end());
117     return g;
118 }
119
120 // --- 7. Clockwise Spiral Traversal ---
121 template <class T>
122 vector<T> spiral_order(const vector<vector<T>> &g) {
123     if (g.empty()) return {};
124     vector<T> order;
125     int top = 0, bot = (int)g.size() - 1;
126     int left = 0, right = (int)g[0].size() - 1;
127
128     while (top <= bot && left <= right) {
129         for (int c = left; c <= right; ++c) order.push_back(
130             ↪ g[top][c]);
131         ++top;
132         for (int r = top; r <= bot; ++r) order.push_back(g[r
133             ↪ ][right]);
134         --right;
135         if (top <= bot) {
136             for (int c = right; c >= left; --c) order.
137                 ↪ push_back(g[bot][c]);
138             --bot;
139         }
140         if (left <= right) {
141             for (int r = bot; r >= top; --r) order.push_back(g
142                 ↪ [r][left]);
143             ++left;
144         }
145     }
146     return order;
147 }
148
149 // --- 8. Generate R x C Spiral Matrix (1 .. R*C) ---
150 vector<vector<int>> generate_spiral(int R, int C) {
151     vector<vector<int>> g(R, vector<int>(C, 0));
152     int val = 1, top = 0, bot = R - 1, left = 0, right = C
153         ↪ - 1;
154     while (top <= bot && left <= right) {
155         for (int c = left; c <= right; ++c) g[top][c] = val
156             ↪ ++;
157         ++top;
158         for (int r = top; r <= bot; ++r) g[r][right] = val
159             ↪ ++;
160         --right;
161         if (top <= bot) {
162             for (int c = right; c >= left; --c) g[bot][c] =
163                 ↪ val++;
164             --bot;
165         }
166         if (left <= right) {
167             for (int r = bot; r >= top; --r) g[r][left] = val
168                 ↪ ++;
169             ++left;
170         }
171     }
172     return g;
173 }
174
175 // --- 9. Neighbor Directions (dr, dc) & Bounds Check
176     ↪ ---
177 constexpr pair<int, int> d4[4] = {{-1, 0}, {0, 1}, {1,
178     ↪ 0}, {0, -1}}; // U, R, D, L

```

```
163 constexpr pair<int, int> d8[8] = {
164     {-1, 0}, {-1, 1}, {0, 1}, {1, 1},
165     {1, 0}, {1, -1}, {0, -1}, {-1, -1}
166 };
167
168 inline bool in_bounds(int r, int c, int R, int C) {
169     return r >= 0 && r < R && c >= 0 && c < C;
170 }
171
172 // =====
173 // Demonstration in main()
174 // =====
175 int main() {
176     vector<vector<int>> mat = {
177         {1, 2, 3},
178         {4, 5, 6}
179     };
180
181     auto print_grid = [](const string &title, const auto &
182         ↪ g) {
183         cout << "--- " << title << " ---\n";
184         for (const auto &row : g) {
185             for (const auto &val : row) cout << setw(3) << val
186                 ↪ ;
187             cout << '\n';
188         }
189     };
190
191     print_grid("Original (2x3)", mat);
192     print_grid("Transpose (3x2)", transpose(mat));
193     print_grid("Rotate 90 CW (3x2)", rotate90_cw(mat));
194     print_grid("Rotate 90 CCW (3x2)", rotate90_ccw(mat));
195     print_grid("Rotate 180 (2x3)", rotate180(mat));
196     print_grid("Flip Horizontal", flip_h(mat));
197     print_grid("Flip Vertical", flip_v(mat));
198
199     auto spiral_mat = generate_spiral(3, 4);
200     print_grid("Generated 3x4 Spiral", spiral_mat);
201
202     auto spiral_vals = spiral_order(spiral_mat);
203     cout << "--- Spiral Traversal Order ---\n";
204     for (int x : spiral_vals) cout << x << ' ';
205     cout << "\n";
206
207     return 0;
208 }
```

5 Text Manipulation, Pattern Matching & Regex

Pattern Finding & Regex Cheatsheet:

Pattern / Regex	Function / Syntax	Complexity / Use
KMP	kmp_search(t, pat)	$O(N + M)$ match
Z-Algorithm	z_search(t, pat)	$O(N + M)$ LCP
Full Match	regex_match(s, re)	Exact validation
Extract All	sregex_iterator	Tokenize / ints
Capture Groups	regex_search(s, m, re)	m[1], m[2]
Replace	regex_replace(s, re, f)	Sanitize string

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // -----
5 // 1. String Manipulation & Parsing Helpers
6 // -----
7
8 // Split string by whitespace
9 vector<string> split_ws(const string &s) {
10     stringstream ss(s);
11     string token;
12     vector<string> res;
13     while (ss >> token) res.push_back(token);
14     return res;
15 }
16
17 // Split string by single character delimiter (e.g., ',',
18 //    or ':')
19 vector<string> split(const string &s, char delim) {
20     vector<string> res;
21     string cur;
22     for (char c : s) {
23         if (c == delim) {
24             res.push_back(cur);
25             cur.clear();
26         } else {
27             cur += c;
28         }
29     }
30     res.push_back(cur);
31     return res;
32 }
33
34 // Split string by a multi-character delimiter
35 vector<string> split(const string &s, const string &
36 //    delim) {
37     vector<string> res;
38     size_t start = 0, end = 0;
39     while ((end = s.find(delim, start)) != string::npos) {
40         res.push_back(s.substr(start, end - start));
41         start = end + delim.length();
42     }
43     res.push_back(s.substr(start));
44     return res;
45 }
46
47 // Join container elements into a string with a
48 //    delimiter
49 template <class T>
50 string join(const vector<T> &v, const string &delim = "
51 //    ") {
52     ostringstream oss;
53     for (size_t i = 0; i < v.size(); ++i) {
54         if (i > 0) oss << delim;
55         oss << v[i];
56     }
57     return oss.str();
58 }
59
60 // Trimming whitespace
61 string ltrim(string s) {
62     s.erase(s.begin(), find_if(s.begin(), s.end(), [](
63 //    unsigned char ch) { return !isspace(ch); }));
64     return s;
65 }
66
67 string rtrim(string s) {
68     s.erase(find_if(s.rbegin(), s.rend(), [](unsigned char
69 //    ch) { return !isspace(ch); }).base(), s.end()
70 //    );
71     return s;
72 }
73
74 string trim(string s) { return ltrim(rtrim(move(s))); }
75
76 // Case conversions
77 string to_lower(string s) {
78     for (char &c : s) c = (char)tolower((unsigned char)c);
79     return s;
80 }
81
82 string to_upper(string s) {
83     for (char &c : s) c = (char)toupper((unsigned char)c);
84     return s;
85 }

```

```

76
77 // Replace all occurrences of 'from' with 'to'
78 string replace_all(string s, const string &from, const
79 //    string &to) {
80     if (from.empty()) return s;
81     size_t pos = 0;
82     while ((pos = s.find(from, pos)) != string::npos) {
83         s.replace(pos, from.length(), to);
84         pos += to.length();
85     }
86     return s;
87 }
88
89 // -----
90 // 2. Linear Pattern Matching: KMP & Z-Algo
91 // -----
92
93 // KMP: Compute Prefix Function (pi[i] = length of
94 //    longest proper prefix of s[0..i] that is also
95 //    suffix)
96 vector<int> prefix_function(const string &s) {
97     int n = (int)s.size();
98     vector<int> pi(n, 0);
99     for (int i = 1; i < n; ++i) {
100         int j = pi[i - 1];
101         while (j > 0 && s[i] != s[j]) j = pi[j - 1];
102         if (s[i] == s[j]) ++j;
103         pi[i] = j;
104     }
105     return pi;
106 }
107
108 // KMP: Find all starting indices of 'pat' in 'text' in
109 //    O(|text| + |pat|)
110 vector<int> kmp_search(const string &text, const string
111 //    &pat) {
112     if (pat.empty()) return {};
113     string s = pat + '#' + text;
114     vector<int> pi = prefix_function(s);
115     vector<int> matches;
116     int m = (int)pat.size();
117     for (int i = m + 1; i < (int)s.size(); ++i) {
118         if (pi[i] == m) {
119             matches.push_back(i - 2 * m); // 0-based start
120             //    index in text
121         }
122     }
123     return matches;
124 }
125
126 // Z-Algorithm: z[i] = longest common prefix between s
127 //    and s[i..n-1] in O(N)
128 vector<int> z_function(const string &s) {
129     int n = (int)s.size();
130     vector<int> z(n, 0);
131     int l = 0, r = 0;
132     for (int i = 1; i < n; ++i) {
133         if (i < r) z[i] = min(r - i, z[i - l]);
134         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
135         if (i + z[i] > r) {
136             l = i;
137             r = i + z[i];
138         }
139     }
140     return z;
141 }
142
143 // Z-Search: Find all starting indices of 'pat' in 'text'
144 //    in O(|text| + |pat|)
145 vector<int> z_search(const string &text, const string &
146 //    pat) {
147     if (pat.empty()) return {};
148     string s = pat + '#' + text;
149     vector<int> z = z_function(s);
150     vector<int> matches;
151     int m = (int)pat.size();
152     for (int i = m + 1; i < (int)s.size(); ++i) {
153         if (z[i] >= m) {
154             matches.push_back(i - (m + 1));
155         }
156     }
157     return matches;
158 }
159
160 // -----
161 // 3. std::regex Recipes for CP
162 // -----
163
164 // Check if entire string matches pattern
165 bool regex_full_match(const string &s, const string &pat
166 //    ) {
167     regex re(pat);
168     return regex_match(s, re);
169 }
170
171 // Extract all numbers (including negative integers)
172 //    from text

```

```

162 vector<long long> extract_all_ints(const string &s) {
163     regex re(R"(-?\d+)");
164     sregex_iterator next(s.begin(), s.end(), re);
165     sregex_iterator end;
166     vector<long long> res;
167     while (next != end) {
168         res.push_back(stoll(next->str()));
169         ++next;
170     }
171     return res;
172 }
173
174 // Extract all words/tokens matching regex
175 vector<string> extract_all_tokens(const string &s, const
    ↳ string &pat = R"(\w+)") {
176     regex re(pat);
177     sregex_iterator next(s.begin(), s.end(), re);
178     sregex_iterator end;
179     vector<string> res;
180     while (next != end) {
181         res.push_back(next->str());
182         ++next;
183     }
184     return res;
185 }
186
187 // Extract capture groups: e.g. "x=12, y=34" with "x=(\d
    ↳ +),\s*y=(\d+)"
188 pair<string, string> extract_pair(const string &s, const
    ↳ string &pat) {
189     regex re(pat);
190     smatch match;
191     if (regex_search(s, match, re) && match.size() >= 3) {
192         return {match[1].str(), match[2].str()};
193     }
194     return {"", ""};
195 }
196
197 // Replace regex matches: e.g. replace consecutive
    ↳ spaces with a single space
198 string regex_sanitize(const string &s, const string &pat
    ↳ = R"(\s+)", const string &fmt = " ") {
199     regex re(pat);
200     return regex_replace(s, re, fmt);
201 }
202
203 // -----
204 // Demonstration & Tests in main()
205 // -----
206 int main() {
207     // 1. Text Parsing
208     string raw = " apple, banana , cherry,date ";
209     cout << "Original: [" << raw << "]\n";
210     cout << "Trimmed:  [" << trim(raw) << "]\n";
211
212     auto parts = split(trim(raw), ',');
213     cout << "Split by comma:\n";
214     for (auto &p : parts) cout << " [" << trim(p) << "]\n
    ↳ ";
215
216     vector<int> nums = {10, 20, 30, 40};
217     cout << "Joined vector: " << join(nums, " -> ") << "\n
    ↳ ";
218
219     // 2. Pattern Matching (KMP & Z-algorithm)
220     string text = "ababcbcabababd";
221     string pat = "ababd";
222     auto kmp_res = kmp_search(text, pat);
223     auto z_res = z_search(text, pat);
224     cout << "\nKMP match index of '" << pat << "' in '" <<
    ↳ text << "': " << join(kmp_res, ", ") << "\n";
225     cout << "Z-algo match index: " << join(z_res, ", ") <<
    ↳ "\n";
226
227     // 3. Regex Extractions
228     string log_entry = "Player 42 scored -15 points at
    ↳ 2026-08-28. Extra data: [x=100, y=250]";
229     cout << "\nLog text: " << log_entry << "\n";
230
231     auto ints = extract_all_ints(log_entry);
232     cout << "Extracted integers: " << join(ints, ", ") <<
    ↳ "\n";
233
234     auto coords = extract_pair(log_entry, R"(x=(\d+),\s*y
    ↳ =(\d+))");
235     cout << "Captured coords: x=" << coords.first << ", y=
    ↳ " << coords.second << "\n";
236
237     string messy = "This has way too many
    ↳ spaces.";
238     cout << "Sanitized string: " << regex_sanitize(messy)
    ↳ << "\n";
239
240     return 0;
241 }

```